```
import pandas as pd

df = pd.read_csv("Retail_dataset (1).csv")

df.head()
```

	Order_ID	Customer	Product	Region	Sales	Quantity
0	1001	Sneha	Laptop	West	1056	2
1	1002	Neha	Tablet	South	904	7
2	1003	Meera	Laptop	South	1597	5
3	1004	Rohan	Keyboard	North	777	8
4	1005	Priya	Phone	South	2028	4

Next steps: Generate code with df New interactive sheet

Q1. Creating a DataFrame

Create a Pandas DataFrame using a Python dictionary with columns: Product, Region, Sales, Quantity. Add at least 5 rows and display it.

```
import pandas as pd

data = {
    "Product": ["Laptop", "Phone", "Tablet", "Monitor", "Keyboard"],
    "Region": ["East", "West", "North", "South", "East"],
    "Sales": [2000, 1500, 1200, 800, 400],
    "Quantity": [5, 10, 7, 3, 15]
}

df1 = pd.DataFrame(data)
df1
```

	Product	Region	Sales	Quantity
0	Laptop	East	2000	5
1	Phone	West	1500	10
2	Tablet	North	1200	7
3	Monitor	South	800	3
4	Keyboard	East	400	15

Next steps: Generate code with df1 New interactive sheet

Explanation

This step involves creating a structured dataset manually using a dictionary. In data analytics, data is often stored in tabular form, and Pandas DataFrame provides an efficient way to organize and manipulate such data.

Each column represents a feature (Product, Region, etc.), and each row represents an observation. This dataset will be useful for performing further analysis such as filtering, aggregation, and statistical operations.

Q2. Data Inspection

Display first 5 rows, last 5 rows, dataset structure, and shape.

```
df.head()
df.tail()
```

```
df.info()
df.shape
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 90 entries, 0 to 89
Data columns (total 6 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Order_ID  90 non-null     int64
 1   Customer  90 non-null     object
 2   Product   90 non-null     object
 3   Region    90 non-null     object
 4   Sales     90 non-null     int64
 5   Quantity  90 non-null     int64
dtypes: int64(3), object(3)
memory usage: 4.3+ KB
(90, 6)
```

Explanation

Data inspection is the first step in any data analysis process. It helps in understanding the dataset before performing any operations.

head() → Quick preview of data tail() → End of dataset check info() → Data types + missing values shape → Size of dataset

This step ensures data quality and helps identify issues like missing values or incorrect data types.

Q3. Column Selection

Select and display Product, Region, and Sales columns.

```
df[["Product", "Region", "Sales"]]
```

	Product	Region	Sales
0	Laptop	West	1056
1	Tablet	South	904
2	Laptop	South	1597
3	Keyboard	North	777
4	Phone	South	2028
...	...	...	...
85	Laptop	East	1330
86	Keyboard	West	741
87	Monitor	East	875
88	Tablet	North	2213
89	Laptop	South	1270

90 rows × 3 columns

Explanation

In real-world analysis, we often work with only relevant columns instead of the entire dataset. This step reduces complexity and focuses on meaningful variables required for analysis.

Q4. Indexing using .loc

Display row with index 10 and Sales value at index 20.

```
df.loc[10]
df.loc[20, "Sales"]
```

```
np.int64(1021)
```

Explanation

.loc is used for label-based indexing. It allows analysts to access specific rows and columns using actual labels.

This is useful when:

You know exact row index You want precise data extraction

Q5. Indexing using .iloc

Display first 6 rows & first 4 columns and Sales of 3rd row.

```
df.iloc[0:6, 0:4]
df.iloc[2]["Sales"]
```

```
np.int64(1597)
```

Explanation

.iloc is position-based indexing. It is useful when working with numerical positions instead of labels.

This method is commonly used in data slicing and feature extraction.

Q6. Row Slicing

Display first 10 rows and rows 15 to 25.

```
df.head(10)
df.iloc[15:26]
```

	Order_ID	Customer	Product	Region	Sales	Quantity
15	1016	Karan	Monitor	West	796	3
16	1017	Neha	Laptop	East	2015	2
17	1018	Anita	Phone	North	1637	3
18	1019	Amit	Phone	East	748	7
19	1020	Neha	Laptop	West	1503	6
20	1021	Priya	Laptop	North	1021	4
21	1022	Raj	Tablet	South	886	7
22	1023	Rahul	Laptop	South	2267	3
23	1024	Karan	Monitor	East	814	7
24	1025	Meera	Monitor	West	1926	6
25	1026	Anita	Tablet	East	839	5

Explanation

Row slicing helps analyze specific portions of data. Instead of processing the entire dataset, analysts often focus on subsets for better understanding and faster computation.

Q7. Filtering Data

Filter:

Region = East Sales > 1500

```
df[df["Region"] == "East"]
df[df["Sales"] > 1500]
```

	Order_ID	Customer	Product	Region	Sales	Quantity
2	1003	Meera	Laptop	South	1597	5
4	1005	Priya	Phone	South	2028	4
9	1010	Amit	Phone	West	1998	4
10	1011	Raj	Monitor	North	1693	6
13	1014	Rohan	Phone	South	2078	3
16	1017	Neha	Laptop	East	2015	2
17	1018	Anita	Phone	North	1637	3
19	1020	Neha	Laptop	West	1503	6
22	1023	Rahul	Laptop	South	2267	3
24	1025	Meera	Monitor	West	1926	6
26	1027	Vikas	Laptop	North	1864	8
27	1028	Vikas	Laptop	North	1551	3
29	1030	Rohan	Monitor	West	1932	5
31	1032	Amit	Laptop	East	2226	6
34	1035	Amit	Tablet	East	2104	3
35	1036	Rohan	Phone	East	2136	8
36	1037	Raj	Phone	East	1650	4
37	1038	Amit	Monitor	East	1567	8
38	1039	Meera	Keyboard	East	2455	3
42	1043	Meera	Keyboard	West	1515	6
43	1044	Vikas	Laptop	West	2248	5
44	1045	Priya	Phone	East	1875	4
49	1050	Karan	Monitor	West	2034	7
52	1053	Rahul	Laptop	East	2276	1
57	1058	Meera	Tablet	North	1666	8
58	1059	Karan	Tablet	West	2404	6
59	1060	Karan	Tablet	West	1651	2
60	1061	Vikas	Monitor	South	1837	8
61	1062	Meera	Keyboard	North	1688	4
63	1064	Pooja	Keyboard	West	2310	5
66	1067	Rahul	Tablet	East	1875	6
69	1070	Rahul	Tablet	West	2134	5
71	1072	Anita	Keyboard	East	1702	3
74	1075	Amit	Monitor	North	2390	5
79	1080	Pooja	Tablet	East	1829	2
81	1082	Anita	Phone	East	1695	7
82	1083	Neha	Keyboard	East	2034	5
88	1089	Raj	Tablet	North	2213	5

Explanation

Filtering is used to extract meaningful insights from data. It allows analysts to:

Focus on specific categories Identify high-performing segments

This is a key step in decision-making.

Q8. Multiple Condition Filtering

Filter rows where:

Region = West Sales > 1200

```
df[(df["Region"] == "West") & (df["Sales"] > 1200)]
```

	Order_ID	Customer	Product	Region	Sales	Quantity
9	1010	Amit	Phone	West	1998	4
19	1020	Neha	Laptop	West	1503	6
24	1025	Meera	Monitor	West	1926	6
29	1030	Rohan	Monitor	West	1932	5
42	1043	Meera	Keyboard	West	1515	6
43	1044	Vikas	Laptop	West	2248	5
49	1050	Karan	Monitor	West	2034	7
55	1056	Rahul	Laptop	West	1481	1
58	1059	Karan	Tablet	West	2404	6
59	1060	Karan	Tablet	West	1651	2
63	1064	Pooja	Keyboard	West	2310	5
68	1069	Priya	Laptop	West	1422	6
69	1070	Rahul	Tablet	West	2134	5
73	1074	Meera	Keyboard	West	1429	7
83	1084	Rohan	Laptop	West	1204	5
84	1085	Neha	Monitor	West	1339	7

Explanation

Multiple condition filtering helps refine analysis further. Using logical operators (&), we can apply more than one condition simultaneously.

This is useful for targeted insights, such as identifying high sales in a specific region.

Q9. Descriptive Statistics

Generate statistics and identify:

Mean sales Max quantity Min sales

```
df.describe()

df["Sales"].mean()
df["Quantity"].max()
df["Sales"].min()
```

```
515
```

Explanation

Descriptive statistics summarize the dataset.

Mean → average performance Max → peak performance Min → lowest performance

This helps analysts understand trends and variability in data.

Q10. Aggregation Analysis

Calculate:

Total sales Average sales Max sales Min sales

```
df["Sales"].sum()
df["Sales"].mean()
df["Sales"].max()
df["Sales"].min()
```

```
515
```

Explanation

Aggregation is used to derive overall insights from data.

Total → overall business performance Average → typical performance Max/Min → extremes

These metrics are essential for reporting and decision-making.